Coding Agents: A Survey of AI-Powered Software Development Tools

February 2026

Note: This document was written as a sample project to demonstrate Clover's capabilities, and was generated with the assistance of a large language model (Claude) via Clover. The content may contain inaccuracies or hallucinations inherent to LLM-generated text.

Abstract

Abstract

Large language models have fundamentally transformed software development, enabling a new generation of AI-powered coding tools that range from inline code completion to fully autonomous software agents. The rapid proliferation of these tools—including GitHub Copilot, ChatGPT, Claude, Cursor, Devin, and SWE-agent—has created a fragmented landscape where researchers and practitioners struggle to understand capabilities, compare systems, and identify research opportunities.

This survey provides a comprehensive overview of AI coding agents from the LLM era (2022–present). We present a taxonomy that categorizes systems along three dimensions: autonomy level (from reactive assistance to autonomous execution), interaction scope (from single-line completion to repository-wide changes), and integration mode (IDE extensions, chat interfaces, CLI tools, and standalone platforms). Using this framework, we systematically analyze four categories of tools: code completion systems, conversational coding assistants, autonomous coding agents, and AI-integrated development environments.

We examine evaluation benchmarks including HumanEval, MBPP, and SWE-bench, discussing their strengths and limitations for measuring real-world utility. Finally, we identify open challenges in reliability, security, code quality, and human-agent collaboration, outlining a research agenda for the software engineering community. Our survey aims to help researchers navigate this rapidly evolving field and identify high-impact research directions.

1 Introduction

The landscape of software development is undergoing a profound transformation. In 2023, GitHub reported that in files where Copilot is enabled, 46% of code is now written with AI assistance [Dohmke, 2023]. Tools like GitHub Copilot have surpassed one million paid subscribers, while autonomous coding agents like Devin and SWE-agent have demonstrated the ability to independently resolve real-world software issues. The emergence of "vibe coding"—where developers describe intent and iterate with AI rather than writing code directly—signals a fundamental shift in how software is created.

Yet this rapid evolution has created significant challenges for researchers and practitioners. The field lacks a unified framework for understanding the capabilities and limitations of different

systems. Questions abound: How do code completion tools differ from conversational assistants? What makes an "agent" autonomous? Which benchmarks meaningfully measure real-world utility? Where should research efforts focus?

1.1 Contributions

This survey addresses these challenges by providing:

- **A comprehensive taxonomy** categorizing coding agents along three dimensions: autonomy level, interaction scope, and integration mode.
- **Systematic coverage** of major systems from the LLM era (2022–present), including code completion tools, conversational assistants, autonomous agents, and integrated development environments.
- **Critical analysis** of evaluation benchmarks, examining what they measure and what they miss.
- **Identification of open challenges** and a research agenda for the software engineering community.

1.2 Scope

We focus on tools that assist with code writing and modification, concentrating on systems from the LLM era (2022–present) that have publicly documented capabilities and evaluation data. We exclude general program synthesis, formal verification tools, and non-LLM static analysis systems.

1.3 Organization

The remainder of this paper is organized as follows. Section 2 provides background on large language models for code and the agent paradigm. Section 3 presents our taxonomy of coding agents. Sections 4–7 survey four categories of tools in detail. Section 8 examines evaluation benchmarks and metrics. Section 9 discusses open challenges and future directions. Section 10 concludes.

2 Background and Foundations

This section provides the technical foundations necessary to understand modern coding agents, covering large language models for code, the transition from models to tools, and the agent paradigm.

2.1 Large Language Models for Code

The foundation of modern coding agents is the large language model (LLM) trained on code. The breakthrough came with OpenAI's Codex [Chen et al., 2021], a GPT model fine-tuned on publicly available code from GitHub. Codex demonstrated that language models could generate functionally correct code from natural language descriptions, achieving 28.8% pass@1 on the HumanEval benchmark.

Subsequent models expanded capabilities and accessibility. CodeGen [Nijkamp et al., 2022] introduced multi-turn program synthesis. StarCoder [Li et al., 2023] provided an open-source alternative trained on permissively licensed code. Code Llama [Rozière et al., 2023] brought code capabilities to the Llama model family with specialized variants for infilling and instruction following.

These models share key capabilities:

- **Code understanding**: Parsing and comprehending existing code
- **Code generation**: Producing code from natural language or partial code
- **Infilling**: Completing code given surrounding context (fill-in-the-middle)

2.2 From Models to Tools

A raw language model is not a usable development tool. The transition requires several enabling components:

Context management: Real codebases exceed model context limits. Tools must select relevant context—open files, imported modules, similar code—to include in prompts.

Prompt engineering: The way requests are formatted significantly impacts output quality. Tools develop specialized prompts for different tasks (completion, explanation, refactoring).

Output processing: Model outputs must be parsed, validated, and integrated into the development environment. This includes handling code formatting, language detection, and error recovery.

Interaction design: Single-turn generation is limiting. Modern tools support multi-turn conversations, iterative refinement, and human-in-the-loop workflows.

2.3 The Agent Paradigm

The most capable coding systems employ an agent architecture. An agent, in this context, is a system that can:

- **Plan**: Decompose complex tasks into steps
- **Use tools**: Execute actions beyond text generation (file I/O, shell commands, web search)
- **Observe**: Process feedback from tool execution
- **Iterate**: Adjust behavior based on results

The ReAct pattern [Yao et al., 2022]—Reasoning and Acting—has been particularly influential. ReAct interleaves reasoning traces ("I need to find the function definition...") with actions (search for function) and observations (search results). This approach enables models to tackle multi-step tasks that require gathering information and adapting to intermediate results.

2.4 Terminology

We adopt the following definitions throughout this survey:

Code completion Real-time, inline suggestions during editing, typically single-line or block-level.

Code generation Creating code from natural language descriptions, often function or file-level.

Coding assistant An interactive system providing conversational help with coding tasks.

Coding agent An autonomous system capable of planning and executing multi-step coding tasks with minimal human intervention.

Agentic coding Human-AI collaborative workflows where agents handle implementation while humans guide and review.

3 A Taxonomy of Coding Agents

To organize the diverse landscape of coding agents, we propose a taxonomy with three dimensions: autonomy level, interaction scope, and integration mode.

3.1 Dimension 1: Autonomy Level

We identify four levels of autonomy:

Level 1 – Reactive The system responds only to explicit user requests. It takes no initiative and makes no suggestions unless prompted.

Level 2 – Proactive The system can suggest actions or improvements, but the human must execute any changes.

Level 3 – Semi-autonomous The system can execute actions, but requires human approval for each step or significant action.

Level 4 – Autonomous The system can plan and execute multi-step tasks independently, seeking human input only for clarification or final review.

3.2 Dimension 2: Interaction Scope

The scope of code that a system can understand and modify:

Single-line Completions and suggestions at the cursor position.

Function/block Generation or modification of complete functions or code blocks.

File Operations spanning an entire file, including refactoring and restructuring.

Repository Cross-file changes, feature implementation, and codebase-wide modifications.

3.3 Dimension 3: Integration Mode

How the system integrates with the developer's workflow:

IDE extension Plugins for existing editors (VS Code, JetBrains, Vim).

Chat interface Web-based or standalone conversation interfaces.

CLI tool Command-line interfaces for terminal-centric workflows.

Standalone platform Complete development environments with integrated AI capabilities.

3.4 System Categories

Using this taxonomy, we identify four primary categories. Table 1 summarizes how each category maps to our three dimensions, where L1–L4 refer to the autonomy levels defined above.

The following sections examine each category in detail.

Table 1: Taxonomy of coding agent categories

Category	Autonomy	Scope	Integration
Code Completion Tools	L1–L2	Single-line–Block	IDE extension
Conversational Assistants	L2–L3	Function–File	Chat interface
Autonomous Agents	L3–L4	File–Repository	CLI, Standalone
Integrated Environments	L1–L4	Single-line–Repository	Standalone platform

4 Code Completion Tools

Code completion tools provide real-time, inline suggestions as developers type. They represent the most widely deployed category of AI coding tools.

4.1 Capability Overview

Modern code completion tools operate with tight latency constraints (typically under 100ms) to provide suggestions that feel instantaneous. They analyze the current file and limited surrounding context to predict what the developer will type next.

Key capabilities include:

- Single-line and multi-line completions
- Fill-in-the-middle (FIM) suggestions
- Context-aware completions using imports and recent edits
- Ghost text preview before acceptance

4.2 Major Systems

GitHub Copilot [GitHub, 2022] launched in 2022 as the first widely-available AI code completion tool, powered initially by Codex and later by custom models. It integrates with VS Code, JetBrains IDEs, and Neovim. Copilot has evolved to include chat capabilities (Copilot Chat) and workspace-level features (Copilot Workspace).

Amazon CodeWhisperer [Amazon Web Services, 2023] provides similar functionality with emphasis on AWS service integration and security scanning. It offers a free tier for individual developers.

Tabnine pioneered AI code completion before the LLM era and has evolved to incorporate large language models. It offers on-premise deployment options for enterprises with strict data requirements.

Codeium provides free AI completion for individuals with a model trained to avoid reproducing licensed code verbatim.

Sourcegraph Cody combines code completion with codebase-aware chat, leveraging Sourcegraph's code intelligence infrastructure.

4.3 Technical Approaches

Fill-in-the-middle (FIM): Rather than left-to-right generation, FIM models are trained to complete code given both prefix and suffix context. This enables more accurate completions mid-function.

Context retrieval: Systems retrieve relevant context from open files, recent edits, and imported modules. The quality of context selection significantly impacts suggestion relevance.

Model architectures: Systems use various models—proprietary (Copilot), open-source fine-tuned (some Tabnine configurations), or custom-trained (Codeium).

4.4 Comparison

Table 2: Comparison of code completion tools

System	Model	Free Tier	On-Premise	Chat
GitHub Copilot	Proprietary	Limited	No	Yes
CodeWhisperer	Proprietary	Yes	No	Yes
Tabnine	Various	Yes	Yes	Yes
Codeium	Custom	Yes	Enterprise	Yes
Sourcegraph Cody	Various	Yes	Enterprise	Yes

4.5 Empirical Findings

GitHub reports that Copilot users complete tasks up to 55% faster [Peng et al., 2023]. Studies show typical acceptance rates of 25–35% for suggestions. However, concerns persist about code quality—research has identified security vulnerabilities in AI-generated code [Pearce et al., 2022] and potential license compliance issues.

4.6 Limitations

Code completion tools fundamentally operate without understanding of broader project goals:

- Cannot ask clarifying questions about intent
- Limited cross-file context
- Quality varies significantly by language and task type
- May suggest code with security vulnerabilities or license issues

5 Conversational Coding Assistants

Conversational coding assistants enable multi-turn dialogue about code. Unlike completion tools, they support open-ended interaction but typically lack direct access to the developer's environment.

5.1 Capability Overview

Conversational assistants can:

- Explain existing code
- Generate functions from descriptions
- Debug issues when given error messages

- Suggest refactoring approaches
- Discuss architecture and design trade-offs

The interaction model requires developers to copy code into the conversation and manually apply suggestions—a significant workflow friction compared to integrated tools.

5.2 Major Systems

ChatGPT (GPT-4, GPT-4o) [OpenAI, 2023] established conversational coding as a mainstream capability. Its broad training enables handling diverse languages, frameworks, and coding styles.

Claude [Anthropic, 2024a] (Claude 3, Claude 3.5 Sonnet) emphasizes longer context windows (up to 200K tokens) enabling analysis of larger codebases within a single conversation.

Gemini [Google DeepMind, 2024] integrates with Google's ecosystem and offers multimodal capabilities for understanding code alongside documentation and diagrams.

Phind specializes in developer queries, combining web search with code generation for up-to-date library and API information.

5.3 Technical Considerations

Context windows: The evolution from 8K to 128K+ token contexts has dramatically expanded what can be discussed in a single conversation. Claude's 200K context enables analyzing entire small codebases.

System prompts: Providers tune models with system prompts that improve coding performance, specify output formatting, and establish behavioral guidelines.

Code formatting: Models must correctly identify programming languages, maintain consistent indentation, and produce syntactically valid code.

5.4 Strengths

Conversational assistants excel at:

- Flexible, exploratory interactions
- Handling ambiguous requests through clarification
- Educational explanations and learning support
- Language and framework-agnostic assistance
- Discussing trade-offs and alternatives

5.5 Limitations

Key limitations include:

- **Workflow friction**: Copy-paste interaction disrupts development flow
- **No environment access**: Cannot see actual file contents or run tests
- **Cannot verify**: Suggestions may not work in context
- **Hallucination**: May fabricate APIs, library functions, or syntax
- **Stale knowledge**: Training cutoffs mean outdated library information

6 Autonomous Coding Agents

Autonomous coding agents represent the frontier of AI-assisted development. Unlike assistants that require manual application of suggestions, these systems can directly read files, write code, execute commands, and iterate on their work.

6.1 Defining Autonomy

An autonomous coding agent has:

- **Environment access**: Can read and write files in an actual codebase
- **Execution capability**: Can run commands, tests, and scripts
- **Feedback loops**: Observes results and iterates on errors
- **Multi-step planning**: Decomposes complex tasks into actionable steps

The key distinction from assistants is operational: agents *act* on code rather than merely *advising* about it.

6.2 Architectural Patterns

Most autonomous agents follow variations of an observe-think-act loop:

1. **Observe**: Gather information (read files, search code, check test results)
2. **Think**: Reason about the task and plan next steps
3. **Act**: Execute an action (edit file, run command)
4. **Repeat**: Process results and continue

Tool use is central to agent capabilities. Common tools include:

- File operations (read, write, search, navigate)
- Shell command execution
- Web search and documentation lookup
- Browser automation for testing

Memory mechanisms help agents maintain context across long tasks:

- Conversation history
- Scratchpads for working notes
- Retrieved context from codebase search

6.3 Major Systems

6.3.1 SWE-agent

SWE-agent [Yang et al., 2024] from Princeton introduces the Agent-Computer Interface (ACI)—a set of custom commands designed for LLM interaction with codebases. Rather than raw shell access, SWE-agent provides commands like `find_file`, `search_dir`, and `edit` with structured output formats optimized for LLM parsing.

Key innovations:

- ACI reduces errors from shell command complexity
- Search/replace editing prevents whole-file regeneration
- Achieves 12.5% on SWE-bench (original) with GPT-4

6.3.2 Devin

Devin [Cognition Labs, 2024] from Cognition Labs is positioned as an "AI software engineer" with access to a complete development environment: shell, browser, and editor. It can execute long-running tasks autonomously, maintaining state across sessions.

Key features:

- Full development environment (not just file access)
- Planning and checkpoint system for complex tasks
- Browser for testing and documentation lookup
- Reported strong SWE-bench performance (proprietary evaluation)

6.3.3 OpenHands

OpenHands [Wang et al., 2024] (formerly OpenDevin) provides an open-source platform for building coding agents. It offers:

- Modular architecture supporting different agent implementations
- Sandboxed execution environment
- Web interface and API access
- Multiple agent implementations for research comparison

6.3.4 Claude Code

Claude Code [Anthropic, 2024b] is Anthropic's CLI-based coding agent. It emphasizes:

- Terminal-native workflow for existing developer tools
- Permission system for controlled autonomy
- Integration with git workflows
- Extended thinking for complex reasoning

6.3.5 Aider

Aider [Gauthier, 2024] takes a git-native approach:

- Automatic commits for each change
- Repository map for efficient context selection
- Architect/editor model separation for planning vs. implementation
- Support for multiple LLM providers

6.3.6 Other Notable Systems

AutoCodeRover [Zhang et al., 2024] uses program structure analysis (AST) rather than text search to locate relevant code, improving precision on bug localization.

Agentless [Xia et al., 2024] challenges the complexity of agent architectures, showing that a simple two-phase approach (localization then repair) can achieve competitive results with lower cost.

6.4 Comparison

Table 3: Comparison of autonomous coding agents

System	Open	Model	Interface	SWE-bench
SWE-agent	Yes	Various	CLI	12–18%
Devin	No	Proprietary	Web	Claimed high
OpenHands	Yes	Various	Web/CLI	15–25%
Claude Code	No	Claude	CLI	Not published
Aider	Yes	Various	CLI	Benchmark varies
AutoCodeRover	Yes	Various	CLI	19–22%
Agentless	Yes	Various	CLI	18–27%

6.5 Common Challenges

Autonomous agents face several persistent challenges:

Long-horizon reliability: Success rates decrease dramatically on tasks requiring many steps. Error cascades are common.

Error recovery: Agents often struggle to recover from mistakes, sometimes entering loops of repeated failed attempts.

Security: Agents with shell access pose security risks. Sandboxing and permission systems are essential but imperfect.

Cost: Agent runs can be expensive, with complex tasks requiring many LLM calls.

7 AI-Integrated Development Environments

A new category of development environments combines completion, chat, and agent capabilities into unified experiences. These tools aim to reduce the friction of switching between separate AI tools.

7.1 The Integration Thesis

Integrated AI IDEs offer several advantages:

- **Rich context**: Access to project structure, open files, terminal, git state
- **Lower friction**: No copy-paste between tools
- **Unified experience**: Completion, chat, and agent modes in one interface
- **Workflow integration**: AI actions integrated with editing, debugging, version control

7.2 Capability Tiers

We observe four tiers of AI capability in modern IDEs:

Tier 1 Inline completion—baseline capability all AI IDEs share

Tier 2 Chat panel with codebase awareness

Tier 3 Multi-file editing and refactoring

Tier 4 Agentic execution (run commands, iterate on tests)

7.3 Major Systems

7.3.1 Cursor

Cursor [Anysphere, 2024] is a VS Code fork with deep AI integration:

- **Composer**: Multi-file editing with diff preview
- **@ mentions**: Fine-grained context control (@file, @codebase, @docs)
- **Agent mode**: Autonomous task execution with terminal access
- **Tab**: Predictive edits based on recent changes

7.3.2 Windsurf

Windsurf (from Codeium) introduces the "Cascade" agent:

- Deep codebase indexing for context retrieval
- Flow-based interaction model
- Autonomous multi-step task execution

7.3.3 GitHub Copilot Workspace

Copilot Workspace [GitHub, 2024] focuses on the issue-to-pull-request workflow:

- Starts from GitHub issue description
- Generates specification and implementation plan
- Proposes multi-file changes
- Integrates with GitHub's review and CI systems

7.3.4 Other Systems

JetBrains AI Assistant integrates AI into IntelliJ-family IDEs with completion, chat, and refactoring support.

Zed AI builds AI capabilities into a new high-performance editor with collaborative features.

7.4 UX Innovations

AI IDEs have pioneered several interface patterns:

- **Diff previews**: Show proposed changes before applying
- **Context control**: @ mentions and file pinning for precise context
- **Approval workflows**: Human checkpoints before file modifications
- **Inline editing**: Apply AI suggestions directly in the editor

7.5 Trade-offs

Integrated environments involve trade-offs:

- **Vendor lock-in**: Leaving means losing AI capabilities
- **Model flexibility**: Some lock users to specific providers
- **Privacy**: Codebase indexing raises data handling questions
- **Learning curve**: New interfaces and workflows to learn

8 Evaluation Benchmarks and Metrics

Evaluating coding agents requires benchmarks that capture real-world utility. This section surveys major benchmarks and discusses their strengths and limitations.

8.1 Function-Level Benchmarks

HumanEval [Chen et al., 2021] contains 164 hand-written Python programming problems. Each problem includes a function signature, docstring, and test cases. The model must generate a function body that passes all tests.

HumanEval+ [Liu et al., 2023a] augments HumanEval with additional test cases to catch solutions that pass original tests through shortcuts or edge case failures.

MBPP [Austin et al., 2021] (Mostly Basic Python Programming) contains 974 crowd-sourced Python problems designed to be solvable by entry-level programmers.

These benchmarks measure basic code generation ability but have significant limitations:

- Problems are short, self-contained, and unrealistic
- No project context or cross-file dependencies
- Focus on algorithmic problems rather than software engineering

8.2 Repository-Level Benchmarks

SWE-bench [Jimenez et al., 2024] represents a paradigm shift in evaluation. It contains 2,294 real GitHub issues from popular Python repositories, paired with the pull requests that resolved them. An agent must:

1. Understand the issue description
2. Navigate an unfamiliar codebase
3. Identify relevant files
4. Generate a patch that resolves the issue
5. Pass the repository's test suite

Variants include:

- **SWE-bench Lite**: 300-problem subset for faster evaluation
- **SWE-bench Verified**: 500 human-verified problems with confirmed solvability

SWE-bench has become the de facto standard for evaluating autonomous coding agents, though it has limitations:

- Focuses primarily on bug fixes
- Python-only
- Test-based evaluation may miss code quality issues

8.3 Other Benchmarks

CodeContests [Li et al., 2022] uses competitive programming problems, testing algorithmic reasoning rather than software engineering.

DS-1000 [Lai et al., 2023] focuses on data science tasks using pandas, numpy, and related libraries.

CrossCodeEval [Ding et al., 2023] evaluates cross-file context utilization.

RepoBench [Liu et al., 2023b] measures repository-level code completion.

8.4 Metrics

pass@k: The probability that at least one of k generated solutions passes all tests. Typically reported as pass@1, pass@10, pass@100.

Resolution rate: For SWE-bench, the percentage of issues where the generated patch passes the test suite.

Cost per task: Emerging metric tracking API costs, relevant for practical deployment.

Table 4: Selected results on major benchmarks (as of early 2025)

Benchmark	Top Performance	System
HumanEval	95%+ pass@1	GPT-4, Claude 3.5
MBPP	85%+ pass@1	Various frontier models
SWE-bench Verified	40–55%	Top agents (2024–2025)

8.5 Current State of the Art

8.6 Limitations of Current Evaluation

Current benchmarks leave significant gaps:

- **Data contamination**: Popular benchmarks may appear in training data
- **Narrow task types**: Most focus on bug fixes, missing feature development
- **Missing dimensions**: Code quality, maintainability, security not measured
- **Real-world gap**: Benchmark success may not predict production utility

8.7 Emerging Approaches

Researchers are exploring:

- Human evaluation studies of AI-assisted development
- A/B testing in production environments
- Multi-dimensional quality assessment (correctness + style + security)
- Task diversity beyond bug fixing

9 Open Challenges and Future Directions

Despite rapid progress, significant challenges remain. We organize these into technical, software engineering, and human factors categories.

9.1 Technical Challenges

9.1.1 Reliability and Consistency

Current agents remain unreliable on complex tasks. Specific issues include:

- **Long-horizon failure**: Success rates decrease dramatically with task complexity
- **Non-determinism**: Same inputs can produce different outputs
- **Error cascades**: Early mistakes compound through multi-step tasks
- **Debugging difficulty**: Agent reasoning is often opaque

9.1.2 Context and Scalability

Large codebases present fundamental challenges:

- Context windows, while growing, still cannot hold entire large projects
- Retrieval quality varies significantly across codebases
- Cross-repository dependencies are poorly handled
- Real-time context updates during editing remain difficult

9.1.3 Verification and Testing

Agents struggle to verify their own work:

- Test generation quality lags code generation
- Agents cannot reliably assess whether changes are correct
- Formal verification integration remains limited

9.2 Software Engineering Challenges

9.2.1 Code Quality

Generated code may be functional but problematic:

- Maintainability and readability concerns
- Inconsistent adherence to project style guides
- Violation of design patterns and idioms
- Technical debt accumulation from AI-generated code

9.2.2 Security

Security risks are significant:

- Vulnerability introduction (common weakness patterns in training data)
- Secret leakage in prompts and context
- Supply chain risks from suggested dependencies
- Insufficient sandboxing of agent execution

9.2.3 Intellectual Property

Legal and ethical concerns persist:

- License compliance in generated code
- Attribution requirements
- Copyright status of AI-generated code
- Training data provenance questions

9.3 Human Factors

9.3.1 Skill Development

Impact on developer learning is unclear:

- Junior developers may miss foundational learning
- Over-reliance risks atrophy of debugging skills
- New skills required (prompt engineering, agent supervision)
- Educational curriculum implications

9.3.2 Workflow Integration

Practical integration challenges:

- Code review of AI-generated code requires new approaches
- Team collaboration patterns with AI tools
- Trust calibration—when to accept vs. verify suggestions
- Organizational policies and governance

9.4 Research Directions

We identify several promising research directions:

Better benchmarks: Evaluation that captures real-world software engineering beyond bug fixes, including feature development, refactoring, and documentation.

Human-agent collaboration: Understanding optimal interaction patterns, trust calibration, and division of labor between humans and agents.

Explainability: Making agent reasoning transparent and debuggable.

Domain-specific agents: Specialized agents for security, testing, DevOps, and specific frameworks.

Multi-agent systems: Coordinating multiple specialized agents for complex software development tasks.

Longitudinal studies: Understanding long-term impact on code quality, developer skills, and team dynamics.

10 Conclusion

This survey has examined the rapidly evolving landscape of AI coding agents, from inline code completion to fully autonomous software development systems.

10.1 Summary of Contributions

We have presented:

- A taxonomy categorizing coding agents by autonomy level, interaction scope, and integration mode

- Systematic coverage of major systems across four categories: code completion tools, conversational assistants, autonomous agents, and integrated development environments
- Analysis of evaluation benchmarks including HumanEval, MBPP, and SWE-bench, with discussion of their limitations
- Identification of open challenges in reliability, security, code quality, and human-agent collaboration

10.2 Key Takeaways

Several themes emerge from our analysis:

Rapid capability growth: The field has progressed from code completion to autonomous agents in approximately two years, with systems now capable of resolving real-world software issues.

Persistent reliability challenges: Despite impressive demonstrations, agents remain unreliable on complex, long-horizon tasks. Error rates that seem acceptable on benchmarks compound in practice.

Evaluation gaps: Current benchmarks, while valuable, do not fully capture real-world utility. Code quality, security, and maintainability remain difficult to measure.

Research opportunities: The intersection of AI and software engineering offers rich opportunities for research in human-AI collaboration, evaluation methodology, and specialized agent development.

10.3 Looking Forward

Coding agents represent a fundamental shift in software development. The tools surveyed here are early steps in a trajectory toward more capable AI assistance. The software engineering research community has a critical role to play—not only in advancing these capabilities, but in ensuring they improve rather than degrade software quality, developer skills, and the craft of programming.

We hope this survey serves as a useful map of the current landscape and a foundation for future research.

References

Amazon Web Services. Amazon codewhisperer. `https://aws.amazon.com/codewhisperer/`, 2023.

Anthropic. Claude 3 model card. `https://www.anthropic.com/claude`, 2024a.

Anthropic. Claude code. `https://claude.ai/code`, 2024b.

Anysphere. Cursor: The ai-first code editor. `https://cursor.com`, 2024.

Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cognition Labs. Introducing devin, the first ai software engineer. `https://www.cognition.ai/blog/introducing-devin`, 2024.

Yangruibo Ding et al. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *NeurIPS*, 2023.

Thomas Dohmke. The economic impact of the ai-powered developer lifecycle and lessons from github copilot. https://github.blog/news-insights/research/the-economic-impact-of-the-ai-powered-developer-lifecycle-and-lessons-from-github-copilot/, 2023. Accessed: 2026-02-28.

Paul Gauthier. Aider: Ai pair programming in your terminal. https://aider.chat, 2024.

GitHub. Github copilot. https://github.com/features/copilot, 2022.

GitHub. Github copilot workspace. https://github.blog/2024-04-29-github-copilot-workspace/, 2024.

Google DeepMind. Gemini: A family of highly capable multimodal models. https://arxiv.org/abs/2312.11805, 2024.

Carlos E Jimenez et al. Swe-bench: Can language models resolve real-world github issues? *ICLR*, 2024.

Yuhang Lai et al. Ds-1000: A natural and reliable benchmark for data science code generation. *ICML*, 2023.

Raymond Li et al. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Yujia Li et al. Competition-level code generation with alphacode. *Science*, 2022.

Jiawei Liu et al. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *NeurIPS*, 2023a.

Tianyang Liu et al. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023b.

Erik Nijkamp et al. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

OpenAI. Gpt-4 technical report. https://arxiv.org/abs/2303.08774, 2023.

Hammond Pearce et al. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *IEEE Symposium on Security and Privacy*, 2022.

Sida Peng et al. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023.

Baptiste Rozière et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Xingyao Wang et al. Opendevin: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Chunqiu Steven Xia et al. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang et al. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Yuntao Zhang et al. Autocoderover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.